

Московский авиационный институт  
(национальный исследовательский университет)

Факультет информационных технологий и прикладной  
математики

Кафедра вычислительной математики и программирования

Лабораторная работа №8 по курсу «Дискретный анализ»

Студент: Д. О. Кочев  
Преподаватель: А. А. Кухтичев  
Группа: М8О-306Б-22  
Дата:  
Оценка:  
Подпись:

Москва, 2024

## Лабораторная работа №8

**Задача:** Заданы длины  $N$  отрезков, необходимо выбрать три таких отрезка, которые образовывали бы треугольник с максимальной площадью.

### Формат ввода

На первой строке находится число  $N$ , за которым следует  $N$  строк с целыми числами-длинами отрезков.

### Формат вывода

Если никакого треугольника из заданных отрезков составить нельзя – 0, в противном случае на первой строке площадь треугольника с тремя знаками после запятой, на второй строке – длины трёх отрезков, составляющих этот треугольник. Длины должны быть отсортированы.

# 1 Описание

Для решения задачи требуется применить жадный лагоритм: будем максимизировать ответ на каждом следующем шаге.

Для начала отсортируем длинны треугольников по убыванию, например, быстрой сортировкой за  $\Theta(n * \log n)$ . Затем, будем читать  $i$ -ю,  $(i + 1)$ -ю и  $(i + 2)$ -ю длинны, и смотреть, можно ли составить их них треугольник. Если можно, то запомним площадь, а если нет, то для следующей итерации просто к  $i$  прибавим 1, так как из  $i$ -й длинны треугольник уже составить будет невозможно. В ответ положим максимальную площадь, которую мы получили, пробегаясь по массиву. Сложность данного решения будет  $\Theta(n + n * \log n)$ , где  $n$  - количество отрезков.

## 2 Исходный код

Мой код получился небольшим и поместился в одну функцию *main*. Чтение длин отрезков и их сортировка

```
1 | int n;
2 | std::cin>>n;
3 | std::vector<int> arr(n, 0);
4 | for (int i = 0; i < n; i++){
5 |     std::cin>>arr[i];
6 | }
7 | std::sort(arr.begin(), arr.end(), decreasingCompare);
```

Затем я реализовываю жадный алгоритм, запомню получившиеся площади и возвращаю максимальную.

```
1 | for(int i = 0; i < n - 2; i++){
2 |     first = arr[i];
3 |     second = arr[i + 1];
4 |     third = arr[i + 2];
5 |     if (third + second > first && (first >= 0 && second >= 0 && third >= 0)){
6 |         double p = double(third + second + first) / 2;
7 |         double s = std::sqrt(p * (p - first) * (p - second) * (p - third));
8 |         if (s > max_s){
9 |             max_s = s;
10 |             max_first = first;
11 |             max_second = second;
12 |             max_third = third;
13 |         }
14 |     }
15 | }
16 | if (n==0 || n == 1 || n == 2 || max_s == 0){
17 |     std::cout<<0;
18 | }
19 | else{
20 |     std::cout<<std::fixed<<std::setprecision(3)<<max_s<<std::endl;
21 |     std::cout<<max_third<<" "<<max_second<<" "<<max_first;
22 | }
```

Ответ после этого цикла будет храниться в переменной  $max_s$ .

| laba8.cpp                            |                                        |
|--------------------------------------|----------------------------------------|
| bool decreasingCompare(int a, int b) | Вспомогательная функция для сортировки |

### 3 Консоль

```
dmitrykochev$ cat test1.txt
4
1
2
3
5
dmitrykochev$ ./laba8 <test1.txt
0
dmitrykochev$ cat test2.txt
7
1
2
3
17
28
0
11
dmitrykochev$ ./laba8 <test2.txt
0
dmitrykochev$ cat test3.txt
5
2
4
5
6
7
dmitrykochev$ ./laba8 <test3.txt
14.697
5 6 7
```

## 4 Тест производительности

Тест производительности представляет из себя следующее: одни и те же тесты мы будем использовать как в моем решении, так и в наивном алгоритме. В наивном алгоритме мы сделаем два вложенных цикла и переберем все варианты. Это решение займет  $\Theta(n^3)$ .

Тестов будет три, они характеризуются исключительно количеством отрезков. В первом тесте  $n = 3 * 10^2$ . Во втором  $n = 10^3$ . В третьем тесте  $n = 3 * 10^3$ .

```
dmitrykochev$ ./benchmark <test1.txt
Greedy algorithm time: 1210us
Native algorithm time: 33900us
dmitrykochev$ ./benchmark <test2.txt
Greedy algorithm time: 2662us
Native algorithm time: 1191883us
dmitrykochev$ ./benchmark <test3.txt
Greedy algorithm time: 5516us
Native algorithm time: 33207704us
```

Как можно заметить, жадный алгоритм с первого теста работает в разы быстрее. Действительно, как говорилось выше, мой алгоритм имеет сложность  $\Theta(n * \log n)$ , а наивный  $\Theta(n^3)$ . Это подтверждает тест производительности.

## 5 Выводы

Выполнив восьмую лабораторную работу по курсу «Дискретный анализ», я узнал, что такое жадные алгоритмы и научился применять их на практике. Данный метод решения мне очень понравился. Он изящен и легок в понимании, но, в то же время, помогает решать сложные задачи и очень сильно их оптимизировать. Убежден, что новые знания и опыт пригодятся мне в будущем.

## Список литературы

- [1] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ, 2-е издание.* — Издательский дом «Вильямс», 2007. Перевод с английского: И. В. Красиков, Н. А. Орехова, В. Н. Романов. — 1296 с. (ISBN 5-8459-0857-4 (рус.))